

AGRIAI 2.0 | FORENSIC SOFTWARE EXAMINATION

Security Threat Model and Findings

Trust boundaries, abuse paths, evidence, impact and concrete remediation

Volume 05

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Founder, designer and sole creator of AgriAI

Computer Science, University of Ghana

Repository examined: ThaddeusHackz/agriai-v2

Evidence commit: 991457272b7fec548e02c645c5593b930af5ea29

Examination date: 18 August 2026

Classification: Internal technical record

Technical basis

This document is grounded in direct source review, route enumeration, dependency audit, secret pattern scanning, strict TypeScript checking, ESLint, production compilation, PostgreSQL integration tests and local HTTP behavior tests. Live third party AI accounts were not available in the examination environment, so provider success is assessed from code paths and contracts rather than a claim of live production availability.

1. Security model

The protected assets are provider quota, admin control, session tokens, chat content, contact and subscriber data, database integrity and agronomic trust. Untrusted actors include anonymous browsers, automated bots, compromised editor accounts, hostile search content and failed or malicious upstream providers. The core trust boundaries are browser to Next.js, Next.js to vendors, Next.js to disk or PostgreSQL, and admin cookie to privileged routes.

2. Trust boundary diagram

```
UNTRUSTED INTERNET
  farmer browser, bot, hostile page, hostile search content
    |
    v
NEXT.JS HTTP EDGE
  public routes | admin routes | session cookie validation
    |
    +--> PROVIDERS: Gemini, Cloudflare, Tavily, weather, TTS, images
    |
    +--> STATE: in-memory document -> db.json -> PostgreSQL JSONB
    |
    +--> ADMIN UI: settings, prompts, users, chats, PII, reset
```

3. Positive controls observed

- * Provider keys are read from server environment variables and obvious placeholders are rejected.
- * Passwords use bcrypt with cost 10. Public user serialization removes passwordHash.
- * Session tokens use crypto.randomBytes(32). Cookies are HttpOnly, SameSite=Lax and Secure in production.
- * Login failure text does not reveal whether the email exists.
- * User management and full database reset require the admin role.
- * Chat, vision, transcription, TTS, search and content forms apply basic length caps.
- * Outbound provider hosts are fixed by code. No general URL fetch endpoint exists.
- * Disk writes use temp-file rename, reducing partial local file corruption.
- * Markdown does not enable raw HTML plugins, reducing direct model-output XSS exposure.
- * No live provider secret was found by the tracked-file pattern and entropy scan.

4. Detailed findings

F-01 | CRITICAL Known fallback admin credential

Evidence: lib/db.ts:166-184 uses admin@agriai.gh and AgriAI@2026Admin when env values are absent. A local production build accepted these credentials and issued an admin cookie.

Impact: A deployment that starts without a strong ADMIN_PASSWORD can be fully taken over. The attacker can read chats and subscribers, change prompts, create users and reset the database.

Engineering action: Remove the password fallback. Fail startup when ADMIN_PASSWORD is missing, placeholder-like or weak. Add a one-time bootstrap command and force rotation. Invalidate all sessions after password changes.

F-02 | HIGH Public chat history object access

Evidence: app/api/history/route.ts exposes GET, POST and DELETE without authentication or ownership proof. A caller who knows sessionId can read, replace or delete that conversation.

Impact: Chat content can be disclosed or destroyed. A malicious client can inject records into the admin message view. Client IDs use Date.now plus Math.random rather than cryptographic capability tokens.

Engineering action: Delete the unused route or bind history to an HttpOnly signed browser token. Use a server-generated random record identifier, ownership check and per-action authorization. Reject arbitrary POST replacement.

F-03 | HIGH No rate limit for paid AI and public writes

Evidence: Chat, vision, generate, transcribe, TTS, health, search, contact, feedback, subscribe, track and history have no shared

quota. Only admin login has an in-memory failure limit.

Impact: Bots can consume provider quota, hold 60-second workers, grow the document and increase disk or PostgreSQL write load. This can become denial of wallet and denial of service.

Engineering action: Apply edge limits by IP and anonymous device token. Add strict route-specific burst, sustained and body-size rules. Require authentication or proof-of-work for the expensive image and audio functions. Enforce vendor budgets.

F-04 | HIGH Known vulnerable dependency snapshot

Evidence: npm audit on 18 August 2026 reports high severity exposure through direct next 16.2.10 and transitive postcss and sharp. It recommends next 16.3.1.

Impact: Impact depends on enabled framework features, but the installed framework is inside multiple published vulnerable ranges. Leaving it unpatched increases avoidable framework-level risk.

Engineering action: Upgrade Next.js and lockfile to a patched release, rerun build and route tests, then audit again. Track GitHub advisories and use automated dependency update pull requests.

F-05 | HIGH Public health route spends quota and returns errors

Evidence: GET /api/health calls Gemini generateContent whenever a key is configured and returns up to 220 characters of thrown error text. The route has no authentication or cache.

Impact: Repeated probes can consume model quota and reveal provider project or request details in error messages.

Engineering action: Make liveness local and constant time. Put provider checks behind admin authorization, cache results for several minutes, redact all vendor text and enforce a strict rate limit.

F-06 | HIGH Full-document growth and multi-instance lost updates

Evidence: All records share one mutable object and one PostgreSQL row. Each process writes the full JSON document. No distributed lock, revision check or transaction prevents two instances from overwriting one another.

Impact: Concurrent instances can lose chats, sessions or settings. Anonymous writes can grow serialization cost and JSONB row size until latency and memory become unsafe.

Engineering action: Move high-growth entities into normalized tables. Use transactions, primary keys, optimistic revisions and bounded retention. Keep settings separate from chats, sessions, analytics events and PII.

F-07 | MEDIUM Editor authorization is broader than labels imply

Evidence: Most admin endpoints check only getSessionUser. Editors can read chats, feedback and subscribers, delete those records, change AI prompts and site settings, edit prices and force PostgreSQL sync. Only users and reset enforce role=admin.

Impact: A compromised lower-privilege editor reaches PII and destructive operations. UI tab hiding is not an authorization control.

Engineering action: Create a permission matrix and central requireRole helper. Limit editors to explicit content actions. Restrict PII, prompt control, deletes, sync and exports to admin or dedicated roles.

F-08 | MEDIUM No explicit CSRF validation

Evidence: State-changing admin routes rely on SameSite=Lax cookies but do not validate Origin, Referer or a CSRF token.

Impact: SameSite reduces common cross-site form attacks but does not replace explicit request provenance, especially under same-site subdomain compromise or browser edge cases.

Engineering action: Validate Origin against the canonical host and add a per-session CSRF token for mutation requests. Keep SameSite and Secure flags as defense in depth.

F-09 | MEDIUM Incomplete browser hardening headers

Evidence: next.config.ts sets nosniff and Referrer-Policy only. Local responses had no CSP, frame-ancestors, Permissions-Policy or HSTS and exposed X-Powered-By.

Impact: The admin surface is less protected against clickjacking and future XSS. Browser capabilities are not constrained.

Engineering action: Add a nonce-based Content-Security-Policy, frame-ancestors 'none', Permissions-Policy, HSTS at the TLS edge and poweredByHeader:false. Inventory required font and vendor origins first.

F-10 | MEDIUM Search prompt injection

Evidence: Tavily, Wikipedia and DuckDuckGo text is concatenated into the model system instruction as CURRENT WEB SEARCH RESULTS. No sanitization, trust label or instruction-isolation protocol is applied.

Impact: A hostile page snippet can direct the model to ignore policy, produce unsafe agronomic advice or misrepresent citations.

Engineering action: Treat retrieved content as quoted data. Delimit it in a lower-trust user context, strip instruction-like markup, use an allowlist of agronomic sources where possible and verify critical recommendations.

F-11 | MEDIUM User-derived TTS is marked public cacheable

Evidence: app/api/tts/route.ts returns Cache-Control: public, max-age=3600 for text that may contain a farmer's private answer.

Impact: A shared intermediary could cache sensitive audio. POST caching is uncommon but the directive is needlessly permissive.

Engineering action: Return Cache-Control: private, no-store. Avoid logging text. Add an explicit privacy statement and short

processing retention at the vendor.

F-12 | MEDIUM Weak media authenticity validation

Evidence: Vision trusts a supplied data URL MIME and approximate character length. Transcription trusts File.type and does not set a minimum size or inspect a file signature.

Impact: Malformed payloads can waste CPU and provider quota. Incorrect MIME labels create unpredictable upstream parsing.

Engineering action: Decode with strict base64 validation, inspect magic bytes, allow known formats, cap decoded bytes and dimensions or duration, and reject polyglot or empty files.

F-13 | MEDIUM Provider fingerprinting

Evidence: /api/config returns provider configured flags and model name. /api/health returns more detailed status and raw provider errors.

Impact: Attackers learn which expensive endpoints are active and which fallback path to target.

Engineering action: Remove provider state from public config. Return only feature availability needed by the UI. Keep detailed diagnostics in authenticated admin telemetry.

F-14 | MEDIUM PII lifecycle and audit gaps

Evidence: Chats, subscriber names and emails, contact messages, session IPs and user agents reside in plaintext application state and PostgreSQL JSONB. No retention worker or tamper-evident admin audit trail exists.

Impact: A storage leak exposes personal content. Admin deletions or prompt changes cannot be attributed or reconstructed.

Engineering action: Define retention by record type, minimize IP and user-agent storage, encrypt backups, isolate roles and create append-only audit events with actor, action, target, before hash and after hash.

F-15 | MEDIUM Unsafe CSV construction

Evidence: The subscriber export joins email, name and timestamp with commas without CSV quoting or spreadsheet formula neutralization.

Impact: Names containing commas break columns. Values beginning with =, +, - or @ can become formulas when opened in spreadsheet software.

Engineering action: Use a tested CSV encoder, quote every field, prefix formula-leading values and set a safe content disposition. Add export tests with adversarial names.

F-16 | LOW Framework fingerprint header

Evidence: The local production response includes X-Powered-By: Next.js.

Impact: This gives automated scanners a fast framework hint. It is minor because client assets already reveal framework traits.

Engineering action: Set poweredByHeader:false in next.config.ts.

F-17 | LOW Limited automated security and route tests

Evidence: The only dedicated repository test script exercises PostgreSQL state. No checked-in tests cover auth, RBAC, history ownership, payload caps, SSE framing or fallback safety.

Impact: Regressions in sensitive route behavior can pass build and type checks.

Engineering action: Add unit, route integration and end-to-end suites. Make tests mandatory in CI and fail on lint errors, high audit findings and changed API contracts.

F-18 | INFO Unverified product metrics

Evidence: lib/db.ts seeds 12,400+ farmers, 92 percent disease accuracy and 35 percent loss reduction. No measurement dataset or evaluation report is present.

Impact: Technical readers may confuse display copy with validated model or impact evidence.

Engineering action: Label these values as targets or provide dated measurement methodology, sample size, confusion matrix, baselines and independent review.

5. STRIDE coverage

Threat	Observed example	Strongest next control
Spoofing	Default admin login and stolen opaque session	No fallback credential, rotation, MFA
Tampering	Public history overwrite and editor prompt change	Ownership and permission matrix, audit log
Repudiation	No actor trail for settings and deletes	Append-only signed audit events
Information disclosure	History IDOR, public health errors, PII document	Authorization, redaction, retention
Denial of service	Unthrottled model calls and full document growth	Edge limits, queue, normalized tables
Elevation of privilege	Broad editor capabilities	Central server-side RBAC

6. Security priority

1. Block default admin access and rotate any deployed credential and session.
2. Remove or secure /api/history.
3. Upgrade Next.js and rerun the complete validation matrix.
4. Place rate limits and body limits in front of every paid or write-heavy route.
5. Make /api/health cheap and private.
6. Separate persistent entities and add retention before scaling.
7. Implement server-side RBAC, CSRF controls and security headers.

Authorship and control statement

This volume was prepared, authored and issued solely under the name of Thaddeus Nii Teiko Tagoe. This threat model is intentionally candid so that Thaddeus Tagoe can defend the system he created before production exposure grows. The repository records identify Thaddeus Tagoe as the founder, sole designer, developer and maintainer of AgriAI.